论文解读 · arXiv 2611.14488

Is Your Robot's Memory Just a Clock?

Temporal Shortcuts in Memory-Augmented VLA Policies

Chubin Zhang, Bo An · Nanyang Technological University · 2026-11

一句话总结 按时间填充的机器人记忆其实在“看钟估进度”：停顿、抖手腕这类什么都没改变的“空事件”会被当成干了活，导致计数任务提前收工；去掉时间戳或做暂停增强都治不好，改为按事件索引记忆才能在不重训的情况下挽回大半损失。

代码： github.com/bennidict23/memory-clock

按时间填充的记忆，能分清“等待”和“干活”吗?

主流记忆设计： FrameSamp

- 从整条轨迹均匀采样 32 帧
- 每帧附加绝对时间步的正弦编码
- 固定成本覆盖整段历史； RoboMME 上最强的发布策略（基于 π0.5 ）

但时间在什么都没发生时也在流逝

真实机器人会等慢规划器、被操作员暂停、等安全监控放行，也会犹豫、空抓。

Null event （空事件）

花了时间、机器人甚至在动，但任务依赖的东西一样没变。理想的事件记忆应当对它“视而不见”。

三种注入的空事件（策略第 48 、 160 步后各一次）

暂停 Pause

原地保持 100 环境步（ 5 s ）；剂量实验用 30 / 300 步

手腕抖动 Fidget

手腕绕保持目标 ±0.3 rad 来回滚转 100 步

空抓 Air-grasp

在静止物体上方抓一下空气，回到事件前姿态

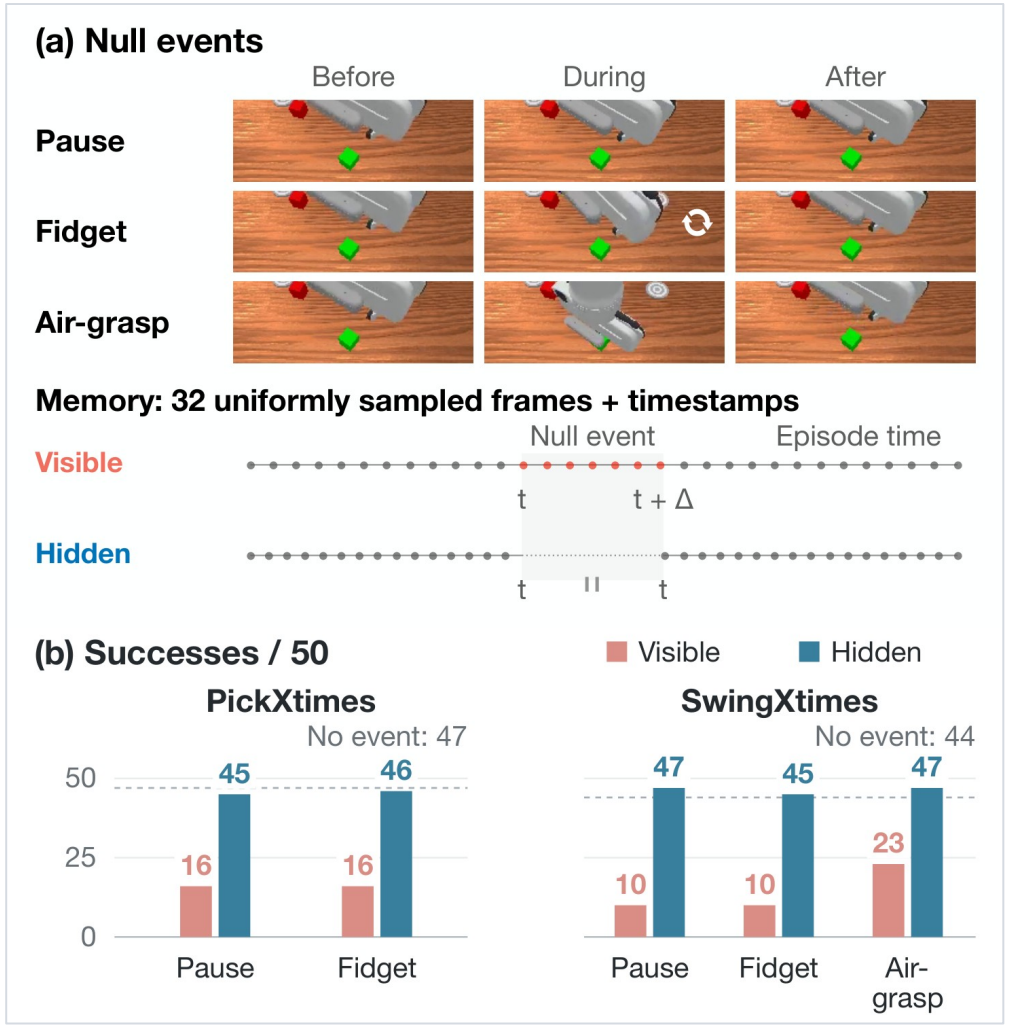
关键对照： Visible vs Hidden

Visible： 事件帧进入记忆，时间戳越过它前进

Hidden： 物理事件完全相同，但帧不写入记忆、时间戳不前进

现象

两次 5 秒暂停让成功率崩溃，而且损伤只通过记忆起作用



PickXtimes · 暂停

47 → 16

隐藏暂停帧后： 45 / 50

SwingXtimes · 暂停

44 → 10

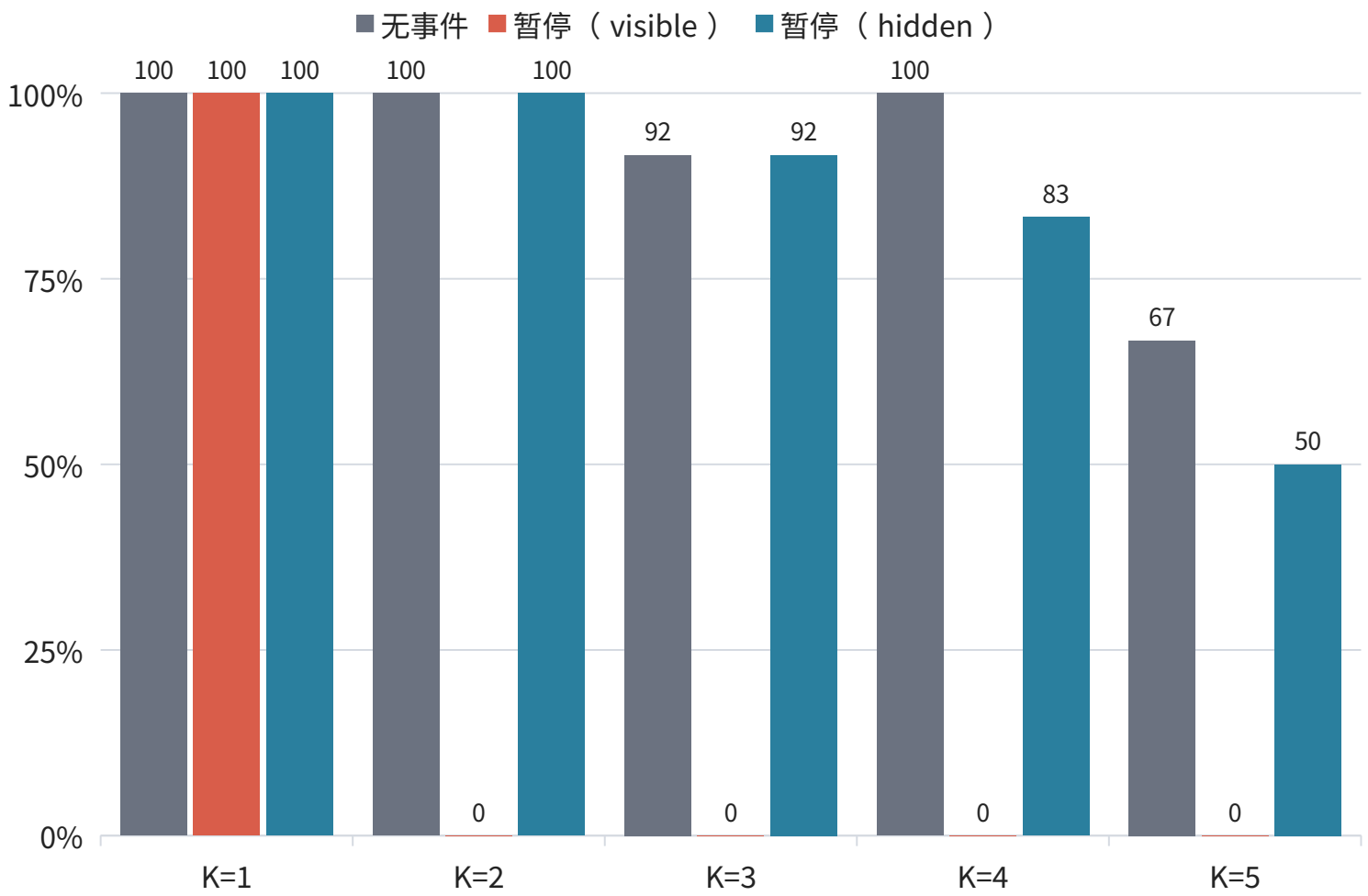
隐藏暂停帧后： 47 / 50

Phantom progress （幻影进度）

抖动同样致命（ 16 / 10 ）；空抓让 SwingXtimes 降到 23 。失败几乎全是在第 K 次之前按下按钮。

现象

只有需要“决定何时停”的 episode 才会失败： K≥2 全军覆没



纵轴：成功率（%）。 episode 数： K=1..5 分别为 16 / 10 / 12 / 6 / 6

K=1 · 暂停

16/16 → 16/16

第一次停的时机本就正确

K≥2 · 暂停 / 抖动

31/34 → 0/34

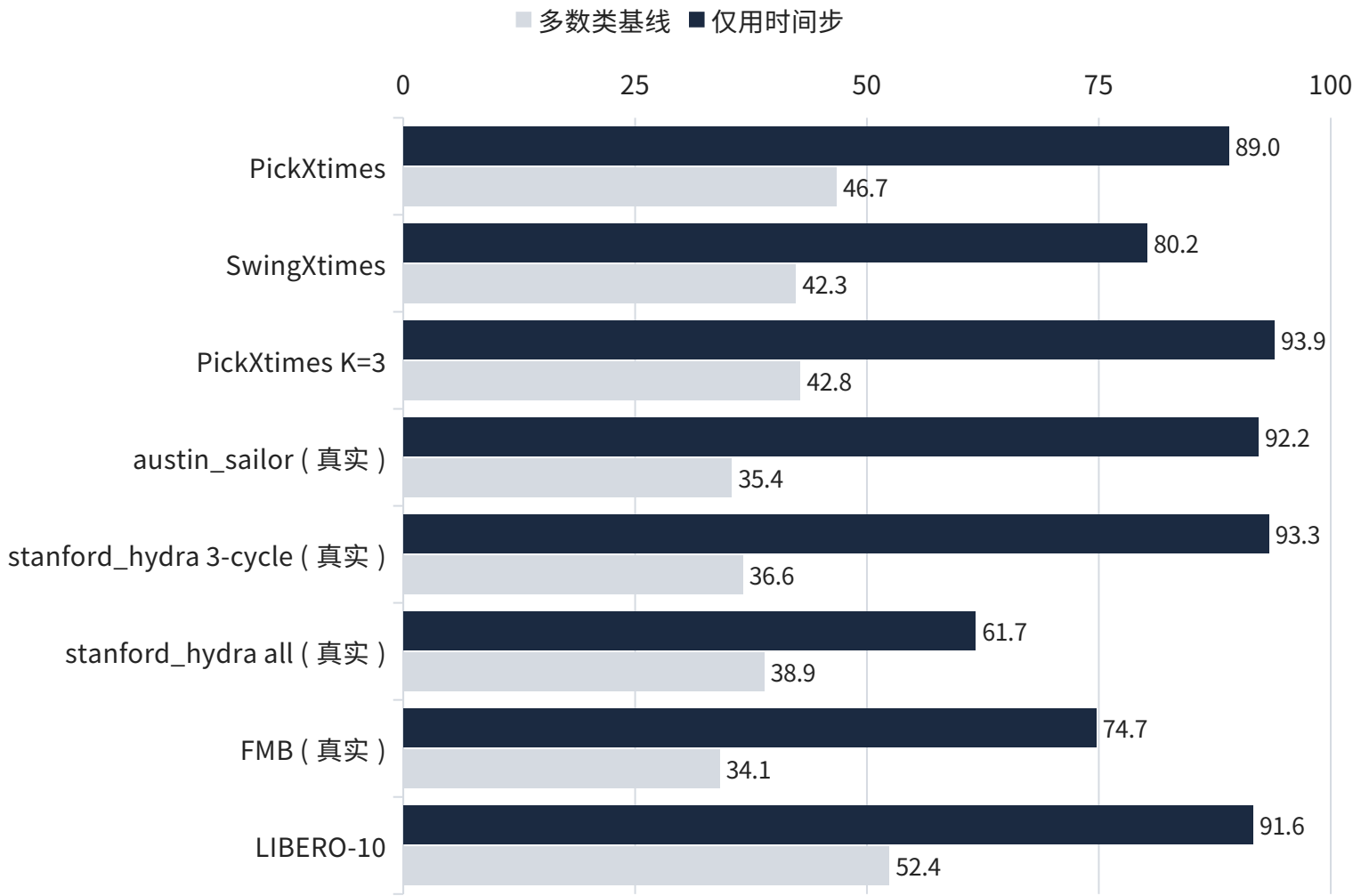
隐藏后回到 29/34

解读

不是“被扰乱”导致的随机失败，而是系统性高估进度：在本该继续时按下按钮。

机制 ①

演示节奏稳定：只看时间步就能猜对进度，模仿学习自然学会“看钟”



PickXtimes 准确率

89%

基线 46.7%

真实遥操作 3-cycle

>92%

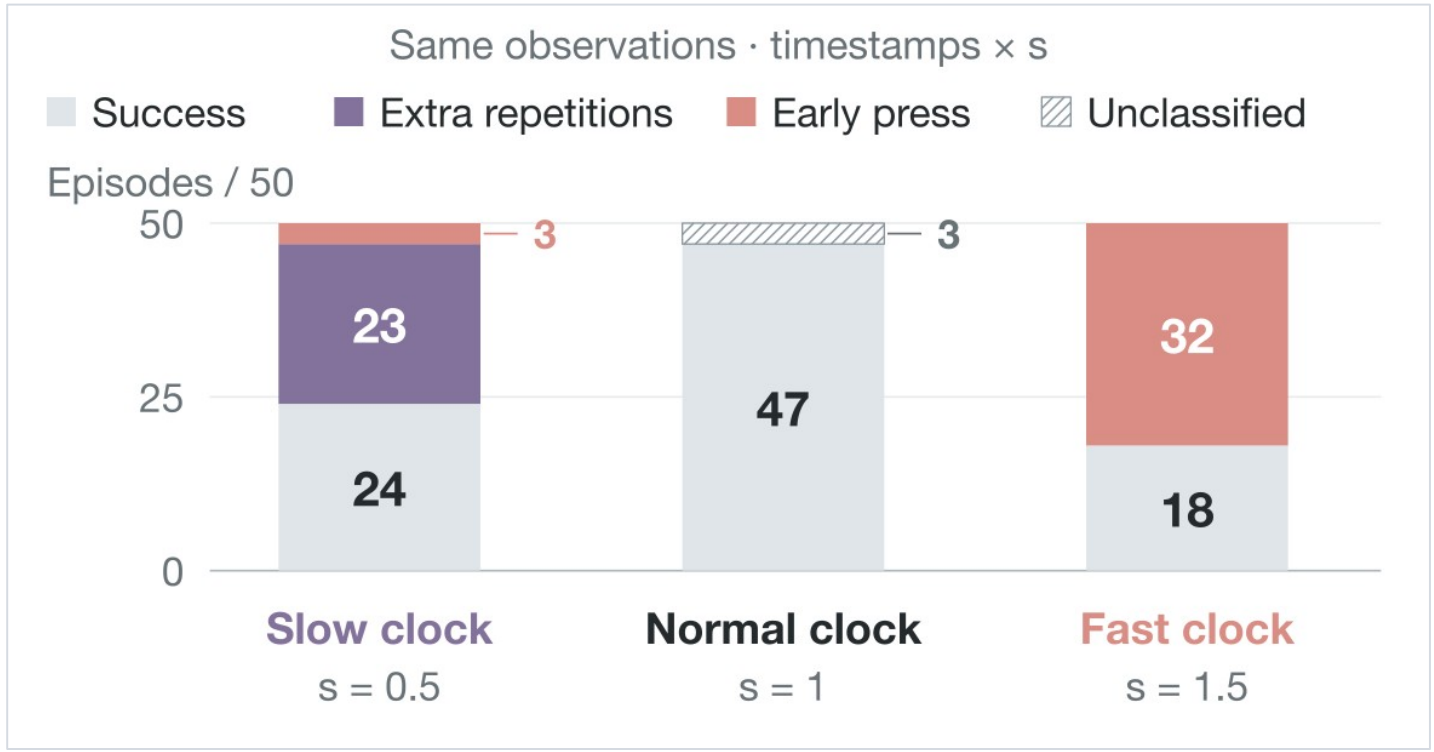
austin_sailor / stanford_hydra

没有反例可学

100 条 PickXtimes 演示里只有 1 条含静止段（12 步）：训练从不惩罚“看钟”。

机制 ②

只缩放时间戳就能操纵失败方向：钟快就早停，钟慢就多做



s = 0.5　慢钟

24 成功

23 次额外重复

s = 1.0　正常

47 成功

基准

s = 1.5　快钟

18 成功

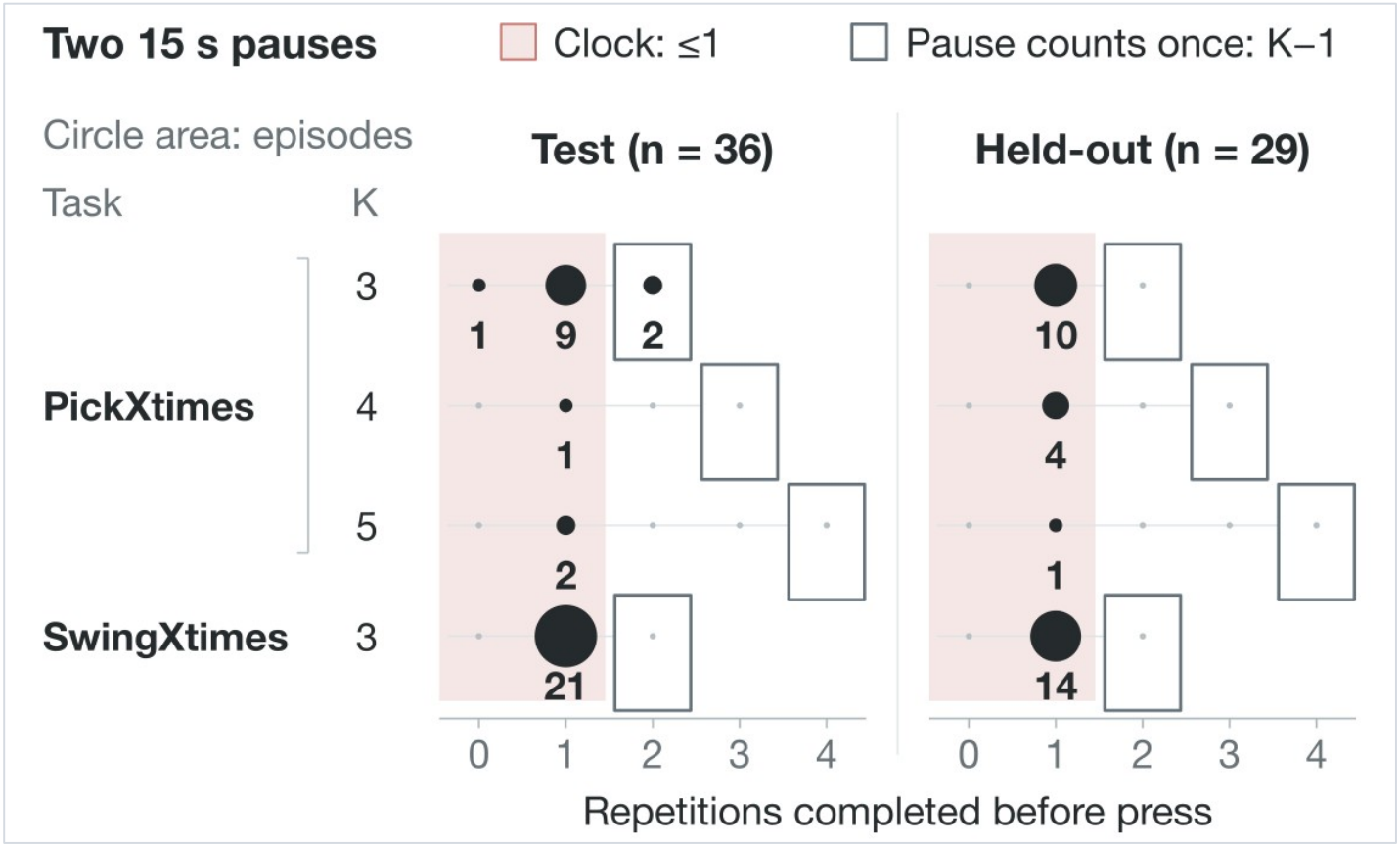
32 次提前按按钮

空事件 = 时钟的局部加速

时间在走、内容不变。翻转在 FrameSamp-Expert 和“冻结时钟”（分布内布局）下同样出现。

机制 ③

策略读的是暂停“有多长”，而不只是“发生过一次事件”



两种解读，两种预测

读时钟： 至多 1 次重复后就按

暂停算一次： K–1 次后才按

测试集

34/36

≤1 次即按

留出集

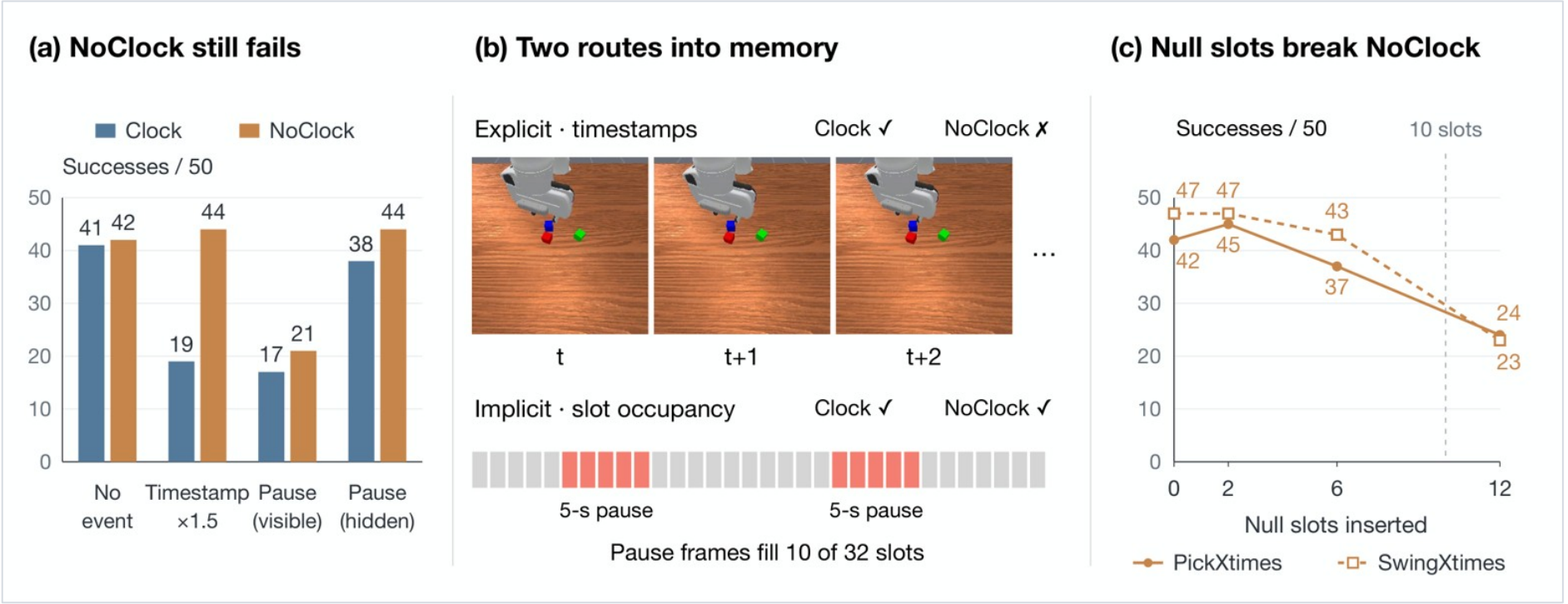
29/29

判据事先固定

时间戳不是唯一通路：只存帧、冻结时间戳也会造成伤害，说明帧本身也携带部分损伤（Sec. E.1）。

机制 ④

去掉时间编码也治不好：空事件的帧照样占着记忆槽位



NoClock 不读时间戳

时间戳 ×1.5： 44 / 50 （Clock： 19）

但暂停照样击穿

PickX 21 / 50， SwingX 14 / 50；隐藏后 44 / 50

空槽越多越糟

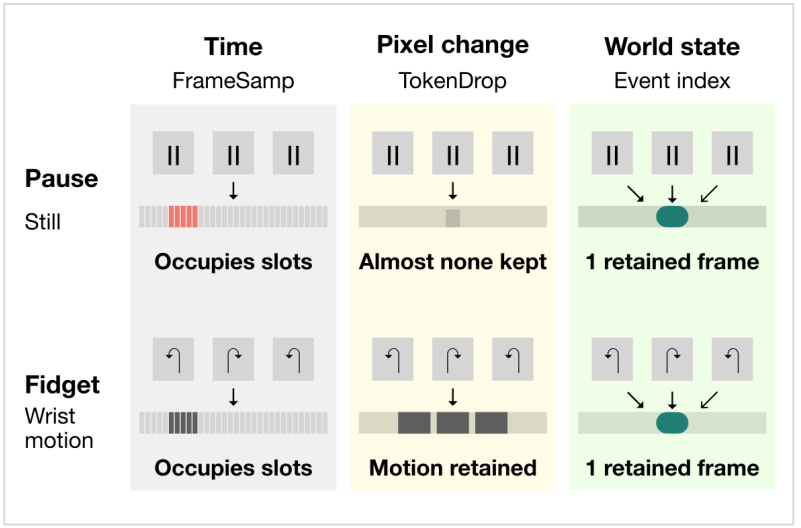
无暂停插入 12 个空槽： PickX 42 → 24

机制 ⑤

脆弱性取决于记忆的索引方式，而不在于有没有记忆

记忆	索引	Pick 暂停	Pick 抖动	Swing 暂停
FrameSamp-Modul	时间	47 → 16	47 → 16	44 → 12
FrameSamp-Expert	时间	44 → 14	44 → 17	49 → 11
FrameSamp-Context	时间	33 → 11	33 → 20	43 → 10
TokenDrop-Modul	像素变化	39 → 30	39 → 21	45 → 27
MemER-style	子目标	38 → 39	38 → 38	24 → 23
Text sub-goals	子目标	46 → 48	46 → 47	–
TTT *	近期	23 → 22	23 → 25	17 → 16
π0.5 无记忆 *	无	18 → 19	18 → 20	15 → 13
SimpleMemVLA †	每步	50 → 47	50 → 38	48 → 36
LTM-π †	每步	50 → 7	50 → 15	50 → 9

* 本身几乎不会计数（K≥2 几乎全败），不受影响不说明问题。† 独立代码库、其他骨干；事件 100 步， SimpleMemVLA 对照为 hidden 。



每种设计都在其索引放行空事件的地方失败

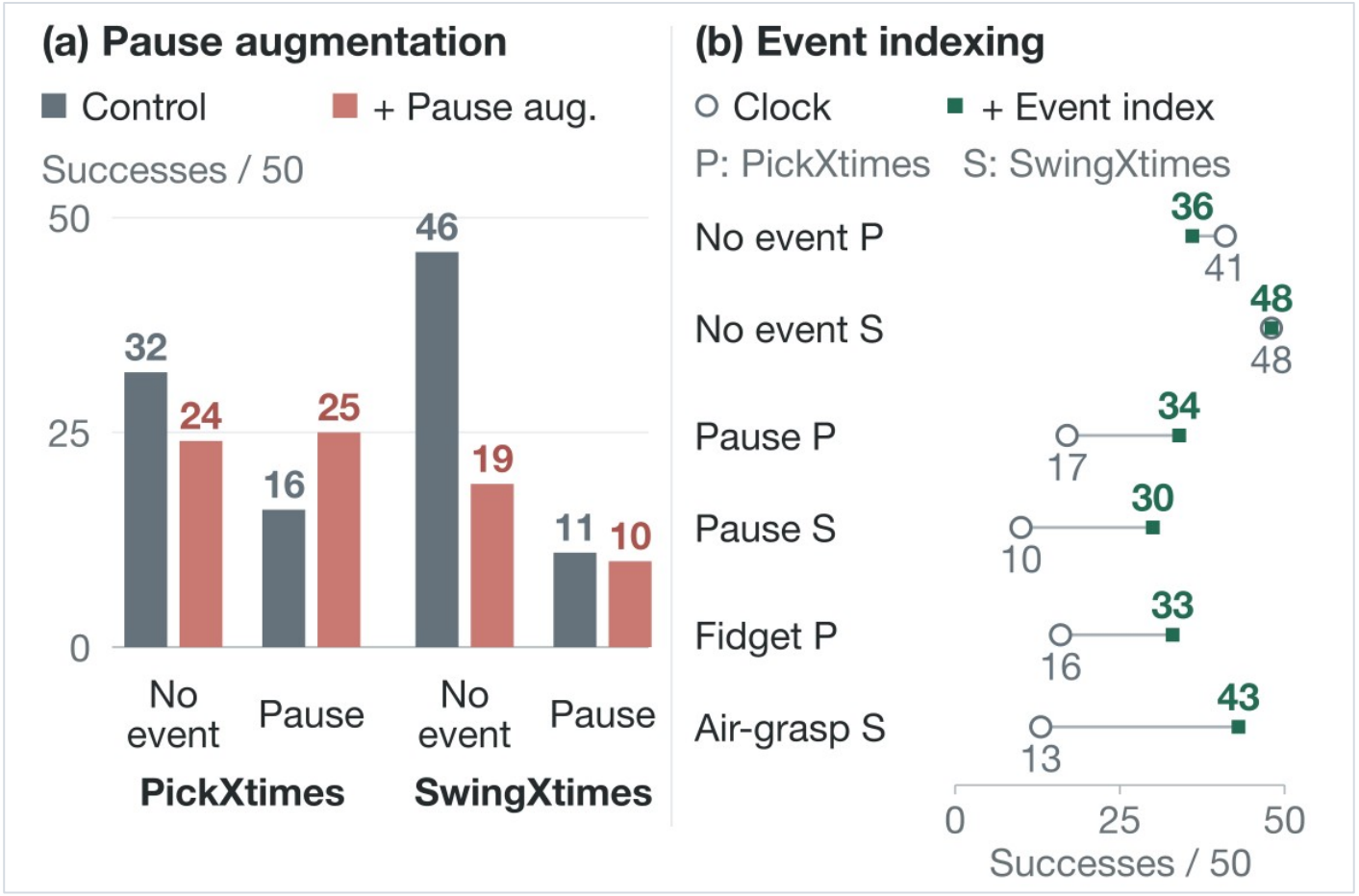
时间： 暂停、抖动都占槽

像素变化： 滤掉静止，放过抖动

子目标： 免疫，但平时计数更弱

修复

改数据没用，改索引有用：事件索引不重训就挽回一半以上损失



改数据：暂停增强 无效

PickX 暂停 16 → 25，但无事件先掉了 32 → 24

SwingX 无事件 46 → 19；暂停 11 → 10

留出集 K≥2： 35/40 → 7/40，计数能力被破坏

改索引：事件索引 有效

暂停： PickX 17 → 34， SwingX 10 → 30

抖动： PickX 16 → 33

空抓： SwingX 13 → 43

代价： PickX 无事件 41 → 36；在留出种子与发布策略上复现

修复

事件索引：只在世界发生变化时推进记忆，也因此有它的盲区

① 闭环检测

构型回到某个已保留帧，场景不变，期间无抓放 / 接触切换

② 压缩

整段压缩为 1 帧；长时间静止只保留前 5 帧

③ 重新计时

时间位置按保留帧重新编号：记忆只随事件推进

信号：机器人轮廓（仿真分割）、夹爪宽度、手臂净外力； 16 个任务共用同一组容差。定位：检验诊断，而非可部署检测器。

同一判据带来的边界

其余 15 个任务总体无代价： 316 vs 317 / 750

误伤真实重复

PickXtimes 中放下方块后回到原状态也是“闭环”：发布策略无事件下有显著代价，并诱发多做一次（ 0/250 → 32/300 ）

不闭环的空事件

单向空抓只能移除约一半；随机微停顿（最贴近部署）几乎救不回来

时长即任务

TimedTransfer / StopCube 中压缩暂停按构造就是错的，这类任务仍需要时钟

外延

同一机制也伤害视频条件任务，基准可能在测“读钟能力”

RouteStick · 两次 15 s 暂停

33 → 7

隐藏时 36；空帧挤掉了演示视频的槽位

视频被挤出记忆

视频与执行共享 32 个时间采样槽位（RouteStick：约 24 → 约 10 槽）

饥饿对照：不暂停、只给视频同样少的槽位，复现伤害（4）

保护对照：暂停但保住视频槽位，恢复（32）

MIKASA-Robo （探索性）

BlinkCount：灯闪 K 次，机器人按按钮 K 次

演示中提示时长与 K 一一对应，区间不重叠

提示时长统一为 50 步：成功 24 → 6，剩余成功集中在常规时长最接近 50 的 K=5

注意：基础成功率仅 48%，未过预设门槛，只作探索性证据

给记忆基准的三种廉价探针

空事件

visible vs hidden 对照

时钟缩放

只改时间戳，看失败方向

固定时长提示

让时长不再泄露答案

评价

因果链条完整、统计严谨；但证据限于仿真，修复更像验证而非方案

优点

- 问题具体，切中真实部署：等待、暂停、犹豫很常见
- 因果链条完整： visible/hidden → 时钟缩放 → NoClock → 跨设计 → 修复
- 统计严谨：配对 McNemar 、 Holm 校正、大量预注册预测，未命中也如实报告
- 给出可迁移的诊断协议，可用于任何记忆策略

局限

- 因果证据全部来自仿真，空事件均为脚本注入；真实数据只证明了相关性
- 剖析集中于 MME-VLA 一个家族；两个独立策略的结果不够整齐
- 事件索引依赖仿真信号，判据与空事件的构造方式相呼应；对随机微停顿基本无效
- 多数从头训练模型只有一个种子；重跑差异可达 12 / 50

结论

记忆应该数事件，而不是数钟声

记录了时间

按时间填充的记忆同时记下“过了多久”和“发生了什么”，并把前者当作进度

两条通路

时间戳携带时长，槽位占用让空事件挤占空间；去掉时钟仍不够

正确的修复点

改索引而非改数据：事件索引不重训即可挽回大半损失

开放问题

如何用真实传感器可靠检测“世界变化”；时长相关任务里时间应作为输入